Scrolling down in the file will display the agent tasks, as shown in listing 4.18. The task descriptions and expected output should be easy to follow. Again, each agent has a specific task to provide better context when working to complete the task.

Listing 4.18 `crewai_coding_crew.py` (task section)

```
code_task = Task(
    description=f"""
You will create a game using python, these are the instructions:
        Instructions
        ------------
        {game}             #1
        You will write the code for the game using python.""",
    expected_output="Your Final answer must be the
↪ full python code, only the python code and nothing else.",
    agent=senior_engineer_agent,
)

qa_task = Task(
    description=f"""You are helping create a game
↪ using python, these are the instructions:
        Instructions
        ------------
        {game}            #1
        Using the code you got, check for errors. Check for logic errors,
        syntax errors, missing imports, variable declarations,
mismatched brackets,
        and security vulnerabilities.""",
    expected_output="Output a list of issues you found in the code.",
    agent=qa_engineer_agent,
)

evaluate_task = Task(
    description=f"""You are helping create a game
↪ using python, these are the instructions:
        Instructions
        ------------
        {game}            #1
        You will look over the code to insure that it is complete and
        does the job that it is supposed to do. """,
    expected_output="Your Final answer must be the
↪ corrected a full python code, only the python code and nothing else.",
    agent=chief_qa_engineer_agent,
)
```

#1 The game instructions are substituted into the prompt using Python formatting.

Finally, we can see how this comes together by going to the bottom of the file, as shown in listing 4.19. This crew configuration is much like what we've seen before. Each agent and task are added, as well as the verbose and process attributes. For this example, we'll continue to use sequential methods.